

GBF异步日志优化

背景

问题现象

US近期主搜迁移GBF，遇到接口计算特别慢（3s+）的问题，当注掉框架中Watcher的对象大小计算后响应回复正常

▼ logWatcher

复制代码

```
...
long resultSize = ObjectSizeCalculator.getObjectSize(result);
    switch (type) {
        case PROCESS:
            TraceLog.info(process, null, null, null, null,
                "success", Type.PROCESS, cost, resultSize);
            break;
    }
...
```

原因分析

ObjectSizeCalculator为JDK中的一个计算类，其原理是通过内存寻址计算hash来达到快速计算对象大小的目标；

之前GBF版本经jmh测试其运行效率较高，对业务影响很小，因此将其保留在主线程执行：

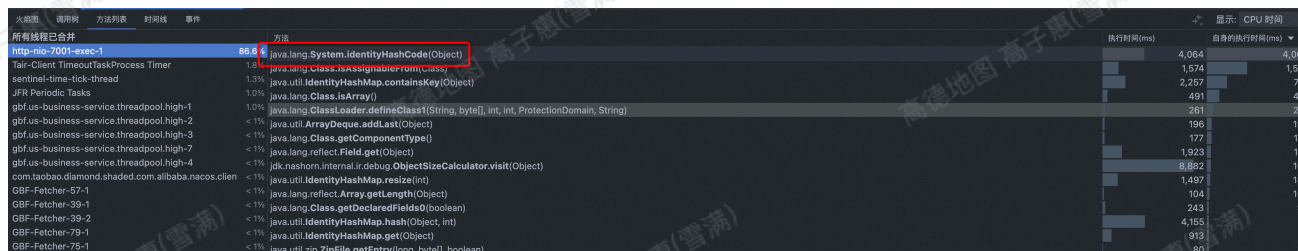
```
/**
 * 结果: objectSizeCalculator性能最优
 * Benchmark
 * ObjectSizeBenchMark.jsonObject      thrpt    2   28.526   ops/ms
 * ObjectSizeBenchMark.objectSizeCalculator thrpt    2   28.988   ops/ms
 * ObjectSizeBenchMark.serializationUtils thrpt    2   26.901   ops/ms
 * ObjectSizeBenchMark.jsonObject      avgt     2    0.703   ms/op
 * ObjectSizeBenchMark.objectSizeCalculator avgt     2    0.547   ms/op
 * ObjectSizeBenchMark.serializationUtils avgt     2    0.572   ms/op
 */
```

但本次迁移中遇到对象中有匿名类对象的情况，将DTO中如下属性删除后计算结果恢复正常

classifyResultDTO.getClassifyVO().getContext().remove("_selectedConfig");

```
classifyResultDTO.getClassifyVO().getContext().remove("_transient");
```

这种属性在内存寻址计算hash时速度将大大降低



上图效率分析中的时间排名，可见System.identityHashCode耗去了绝大部分时间。

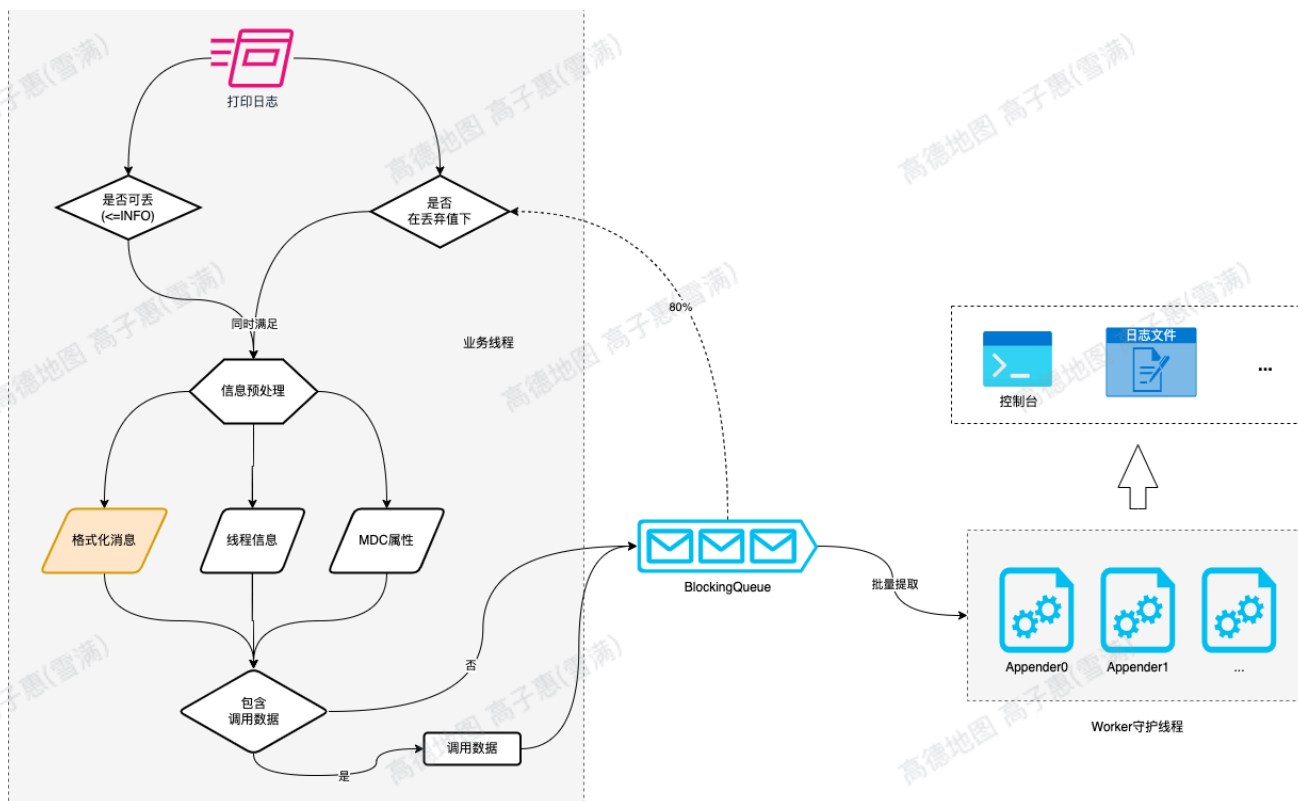
了解后得知此为原有业务代码迁移带过来的逻辑，主要是为了将某些业务数据加工逻辑向后传递复用；但这样放在Node结果通过上下文传递显然不太合适，我们对Node结果的定义为承载业务属性的DTO对象而非某段业务逻辑。

调查结论

1. 针对问题案例，我们已对代码中不合适的写法进行了更正；
2. 同时考虑打印trace日志及其相关的计算逻辑是否可从业务线程中剥离出来，通过解耦提升业务的执行效率。

设计

技术调研



上图为框架所用LogBack日志组件的AsyncAppender打印过程，从中可知：

- AsyncAppender中有一个守护线程Worker它负责实际的日志打印工作
 - 打印前会执行LogEvent的预处理方法
 - 循环执行appender列表的打印逻辑
- 打印线程通过BlockingQueue获取从业务线程添加的打印日志
 - 队列可设置丢弃值，日志堆积时将会抛弃多的日志
 - 当前默认80%的阈值，满时丢弃INFO及以下日志
- 业务线程打印日志时将会调用Appender的方法
 - 判断日志队列的状态，决定是否丢弃日志
 - 执行预处理加工日志信息
 - 格式化后的消息（其中Object会调用其toString方法）
 - 打印日志的线程信息
 - MDC属性表
 - 如开，打印调用数据（CallerData）

概要设计

方案分析

异步日志的优化主要考虑三个方案：

1. 直接用线程（池）执行打印日志及其相关代码逻辑

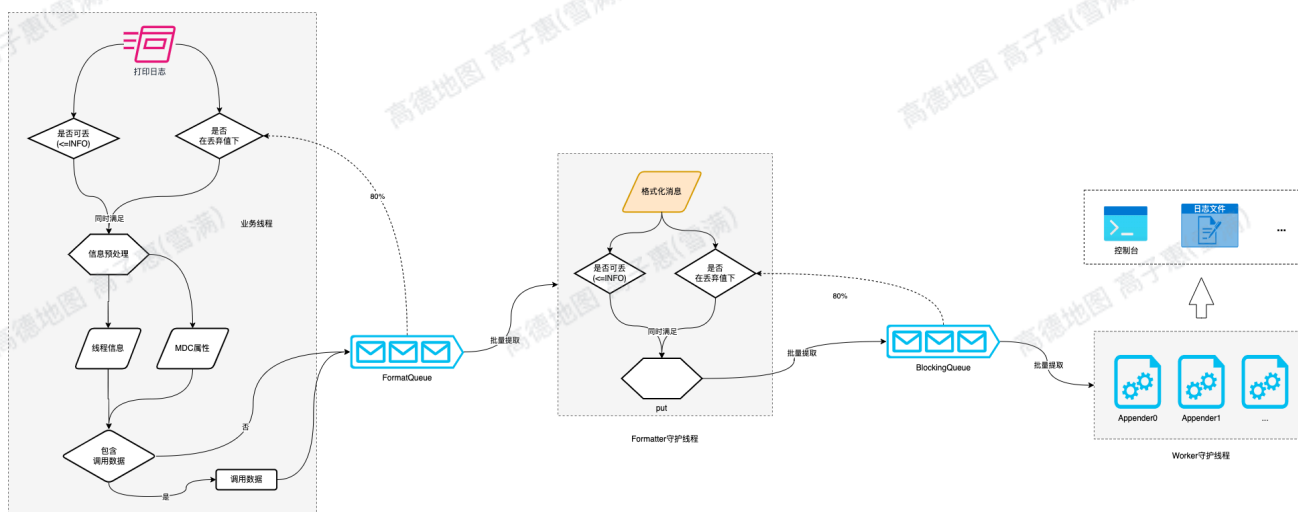
2. 继承改造AsyncAppender将需要的执行逻辑分离到Appender中执行
3. 直接继承屏蔽掉主线程执行预处理时的代码，将其延后到Worker线程执行

三个方案各有如下利弊

方案	单线程	线程池	Appender改造	屏蔽预处理
优点	- 简单易实现 - 复杂度较低	- 计算性能高 - 复杂度最高	- 复用框架部分逻辑 - 复杂度中等 - 二级队列组合使用效率和吞吐两全	复杂度最低
缺点	- 单独用吞吐能力有限 - 如配合队列用则与Appender改造无异	为维护日志的顺序性需维护一个有序队列来往下传日志，方案复杂化	暂无	Worker线程执行日志打印是一个IO过程，将其与业务计算逻辑放在同线程执行效率降低

综合考虑：采用Appender改造的方案

整体设计



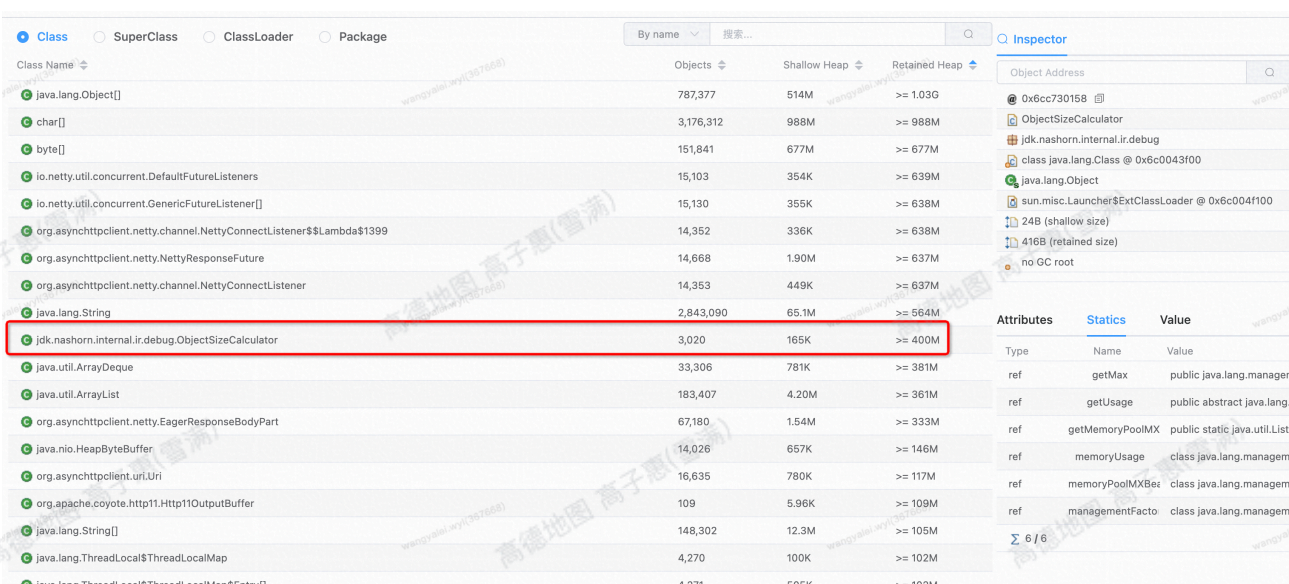
如上图所示，新的优化方案在Logback框架的AsyncAppenderBase基础上扩展出了一组队列和守护线程，将比较耗时的日志格式化延迟到了其中，这样原有的日志流过程从业务->格式化日志->队列->

打印线程；变成了业务->队列->格式化线程->格式化日志->队列->打印线程；以此实现了日志逻辑与业务执行的解耦。

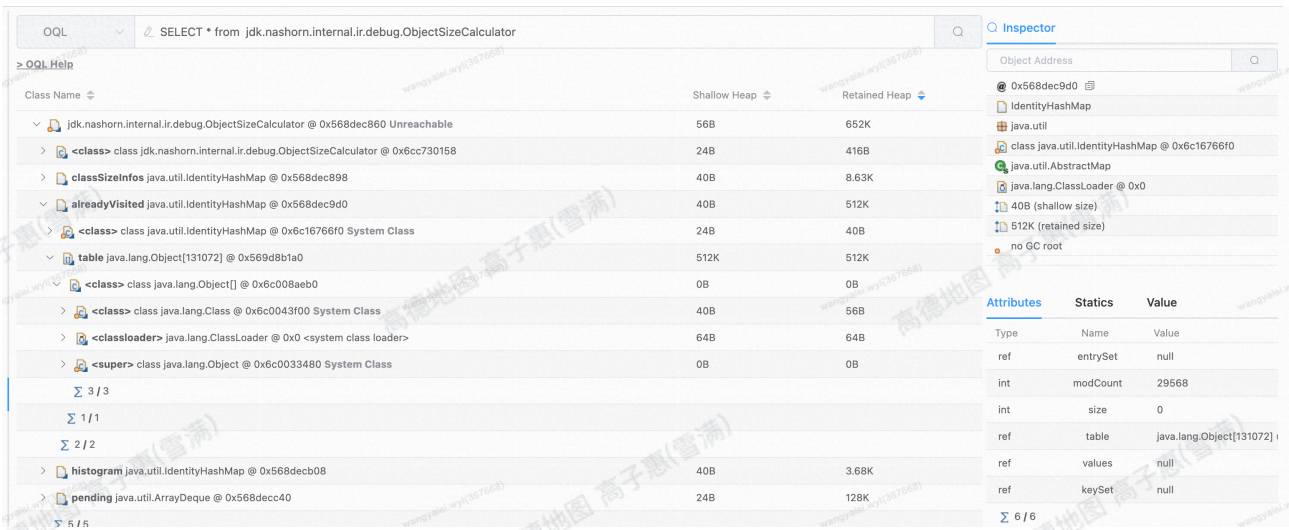
执行过程

其中ObjectSizeCalculator原有方法为每次计算会new一个新的对象来执行；

分析线上机器其确实占用了一定的内存空间：



Class Name	Objects	Shallow Heap	Retained Heap
java.lang.Object[]	787,377	514M	>= 1.03G
char[]	3,176,312	988M	>= 988M
byte[]	151,841	677M	>= 677M
io.netty.util.concurrent.DefaultFutureListeners	15,103	354K	>= 639M
io.netty.util.concurrent.GenericFutureListener[]	15,130	355K	>= 638M
org.asynchttpclient.netty.channel.NettyConnectListener\$\$Lambda\$1399	14,352	336K	>= 638M
org.asynchttpclient.netty.NettyResponseFuture	14,668	1.90M	>= 637M
org.asynchttpclient.netty.channel.NettyConnectListener	14,353	449K	>= 637M
java.lang.String	2,843,090	65.1M	>= 564M
jdk.nashorn.internal.ir.debug.ObjectSizeCalculator	3,020	165K	>= 400M
java.util.ArrayDeque	33,306	781K	>= 381M
java.util.ArrayList	183,407	4.20M	>= 361M
org.asynchttpclient.netty.EagerResponseBodyPart	67,180	1.54M	>= 333M
java.nio.HeapByteBuffer	14,026	657K	>= 146M
org.asynchttpclient.uri.Uri	16,635	780K	>= 117M
org.apache.coyote.http11.Http11OutputBuffer	109	5.96K	>= 109M
java.lang.String[]	148,302	12.3M	>= 105M
java.lang.ThreadLocal\$ThreadLocalMap	4,270	100K	>= 102M
java.lang.ThreadLocal\$ThreadLocalMap\$Entry[]	4,271	505K	>= 102M

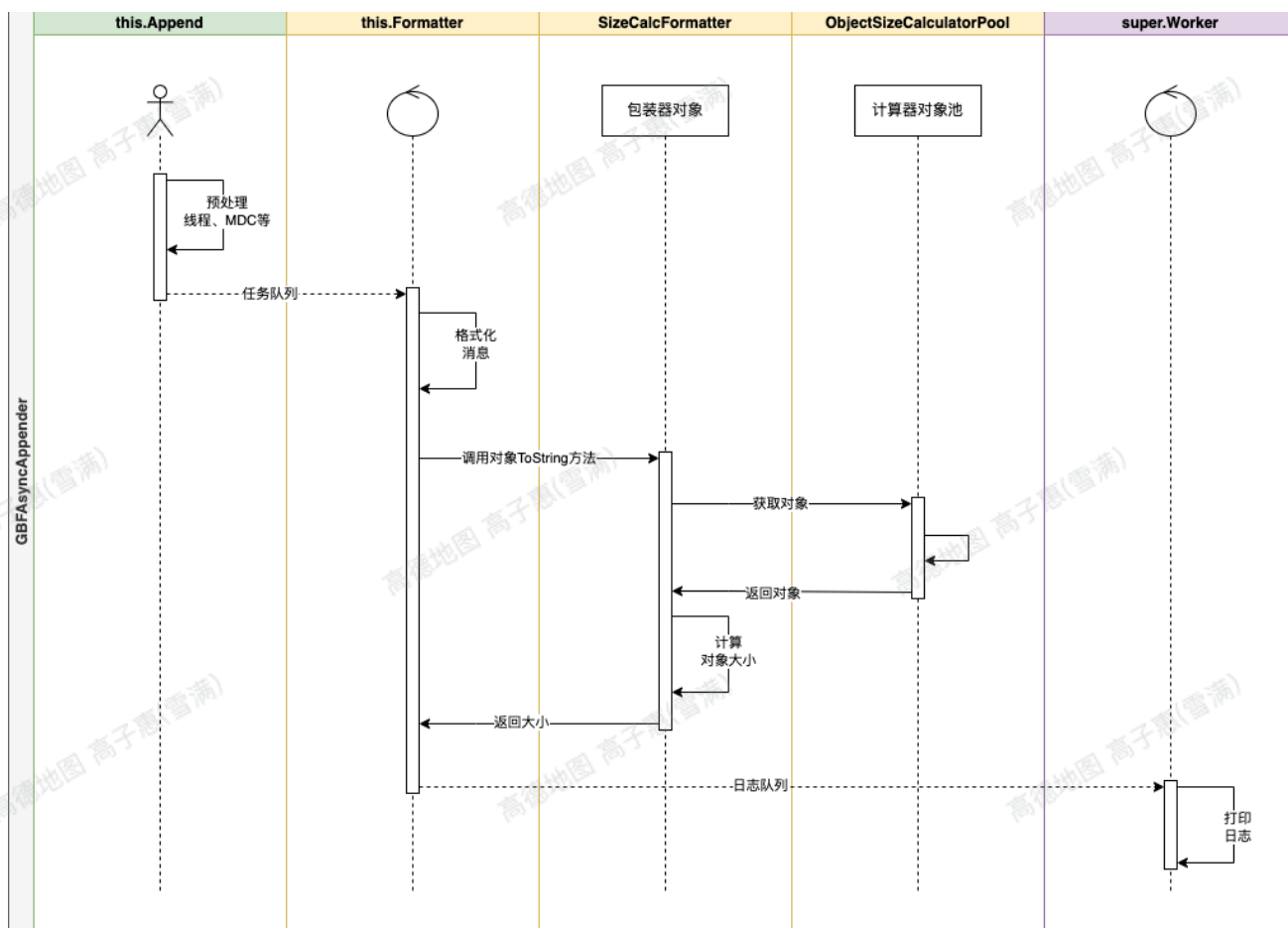


Field Name	Value
entrySet	null
modCount	29568
size	0
table	java.lang.Object[131072]
values	null
keySet	null

在原来的执行逻辑中，对象计算存在于业务线程中，有可能是并行的，所以每次计算都new一个对象是合理的；

但本次改造后对象的计算过程已放入独立的守护线程，串行执行；

经过分析代码发现其可以优化为固定对象池，每次计算可复用，对象池上限与计算队列相同，具体过程如下：

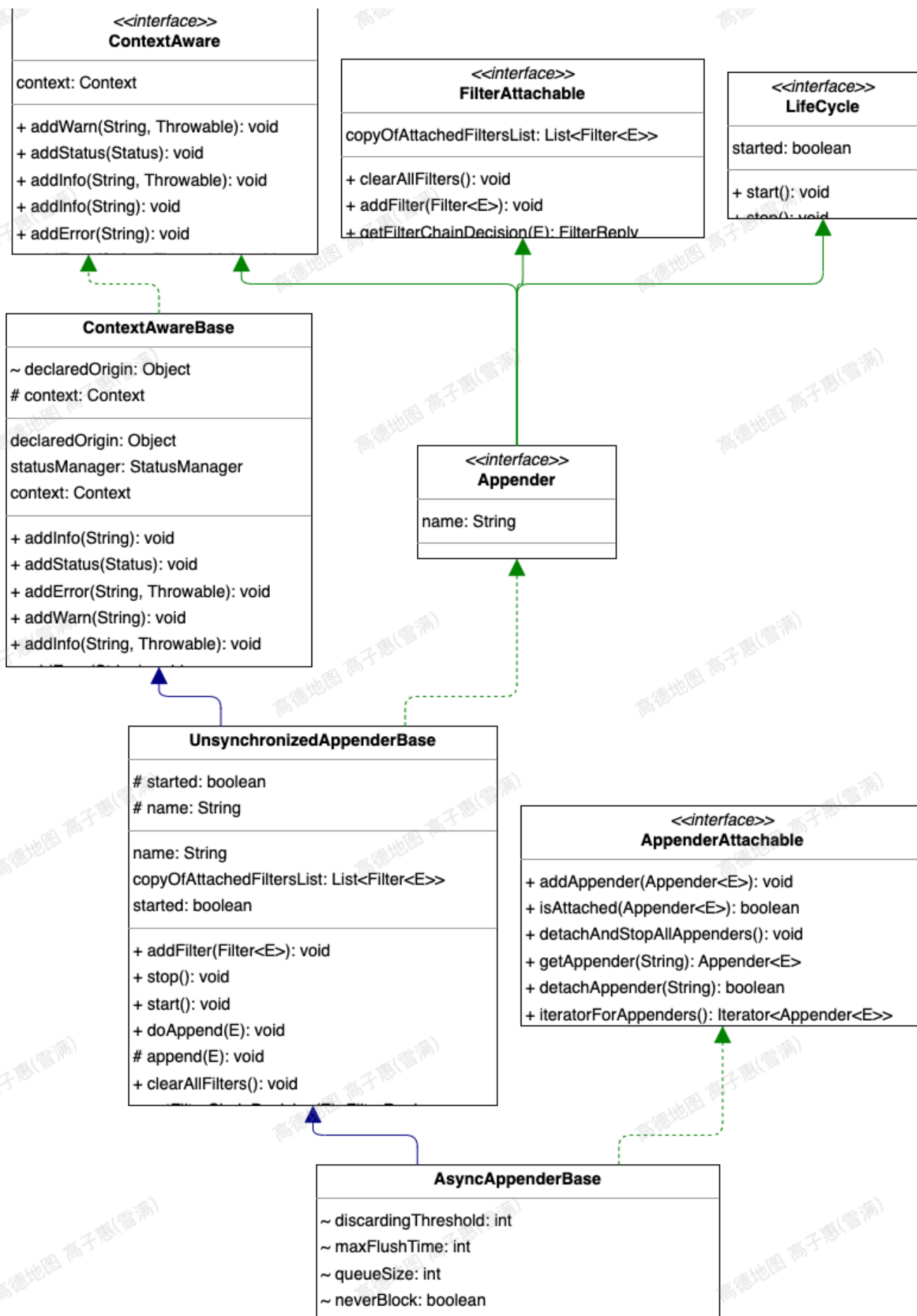


进一步，分析日志的执行线程和格式化过程，均为单队列、单线程、顺序循环执行；同时每次计算时计算器均会清理上一次的数据：

0 个用法

```
public synchronized long calculateObjectSize(Object obj) {  
    this.histogram.clear();  
  
    try {  
        Object o = obj;  
  
        while (true) {  
            this.visit(o);  
            if (this.pending.isEmpty()) {  
                long var3 = this.size;  
                return var3;  
            }  
  
            o = this.pending.removeFirst();  
        }  
    } finally {  
        this.alreadyVisited.clear();  
        this.pending.clear();  
        this.size = 0L;  
    }  
}
```

因此我们始终用一个计算器对象即可，进一步对计算器的生命周期进行优化，改为全局静态对象，执行过程如下：



```

maxFlushTime: int
neverBlock: boolean
remainingCapacity: int
discardingThreshold: int
numberOfElementsInQueue: int
queueBelowDiscardingThreshold: boolean
queueSize: int

+ getAppender(String): Appender<E>
+ iteratorForAppenders(): Iterator<Appender<E>>
# preprocess(E): void
+ isAttached(Appender<E>): boolean
+ detachAppender(String): boolean
+ detachAppender(Appender<E>): boolean
- putUninterruptibly(E): void
+ detachAndStopAllAppenders(): void
# append(E): void
+ stop(): void
# isDiscardable(E): boolean
- put(E): void

```

```

GBFAsyncAppender
~ includeCallerData: boolean
includeCallerData: boolean

+ appendSuper(ILoggingEvent): void
# append(ILoggingEvent): void
+ stop(): void
# isDiscardable(ILoggingEvent): boolean

```

其中用于格式化消息的守护线程：

✓ 格式化线程

复制代码

```

/**
 * 用于将预处理阶段和业务线程解耦提效
 */
class Formatter extends Thread {
    public void run() {
        GBFAsyncAppender parent = GBFAsyncAppender.this;

        // 启动后循环处理
        while (parent.isStarted()) {
            try {
                List<ILoggingEvent> elements = new ArrayList<>();
                ILoggingEvent e0 = parent.formatQueue.take();

```

```

        elements.add(e0);
        parent.formatQueue.drainTo(elements);
        // 加入父类的队列打印日志
        elements.forEach(parent::appendSuper);
    } catch (InterruptedException e1) {
        // exit if interrupted
        break;
    }
}

addInfo("Formatter thread will flush remaining events before
exiting.");

for (ILoggingEvent e : parent.formatQueue) {
    parent.appendSuper(e);
    if (!parent.formatQueue.remove(e)) {
        addWarn("GbfAsyncAppender Failed to remove event from
formatQueue: " + e.getFormattedMessage() + ".");
    }
}
}
}

```

其中appendSuper方法会执行消息格式化以及加入二级队列

▽ 进一步处理

📄 复制代码

```

public void appendSuper(ILoggingEvent eventObject) {
    if (!isStarted()) {
        addError("GbfAsyncAppender is not started.");
        return;
    }
}

```

```
}  
// 格式化日志  
eventObject.getFormattedMessage();  
super.append(eventObject);  
}
```

SizeCalcFormatter

▼ 计算器

📄 复制代码

```
/**  
 * 对象计算器  
 * @author wangyalei  
 * @since 2.3.9  
 */  
@AllArgsConstructor  
public class SizeCalcFormatter {  
    private final static ObjectSizeCalculator CALCULATOR = new  
ObjectSizeCalculator(ObjectSizeCalculator.getEffectiveMemoryLayoutSpecification  
());  
  
    private Object object;  
  
    public static SizeCalcFormatter of(Object object) {  
        return new SizeCalcFormatter(object);  
    }  
  
    @Override  
    public String toString() {  
        try {  
            long size = null == this.object ? 0L :  
CALCULATOR.calculateObjectSize(this.object);  
            return String.valueOf(size);  
        } catch (Exception e) {  
            ErrorLog.error("SizeCalcFormatter.toString error: ", e);  
        }  
        return "";  
    }  
}
```

验证

复制代码

新					
Benchmark	Mode	Cnt	Score	Units	
AsyncLogBenchMarkTest.asyncLog	thrpt	3	2998.812	ops/ms	
AsyncLogBenchMarkTest.asyncLog	avgt	3	0.006	ms/op	
旧					
Benchmark	Mode	Cnt	Score	Units	
AsyncLogBenchMarkTest.asyncLog	thrpt	3	42.380	ops/ms	
AsyncLogBenchMarkTest.asyncLog	avgt	3	0.417	ms/op	

可见，新方案比原有方案在执行效率上快了许多。

其他

测试建议

- 对服务接口进行压测
- 测试服务的RT进行对比
- 查看新旧方案的日志收集条数（有无堆积丢失）

参考文档

logback: [qos-ch/logback: The reliable, generic, fast and flexible logging framework for Java. \(github.com\)](https://qos-ch.github.io/logback/)